

SistemaAsistencia

Como funciona el sistema entero

Aplicativo de escritorio (.NET WinForms) + API (Node/Express) + App movil
(React Native / Expo)

Documento explicativo - Septiembre 2026

Indice

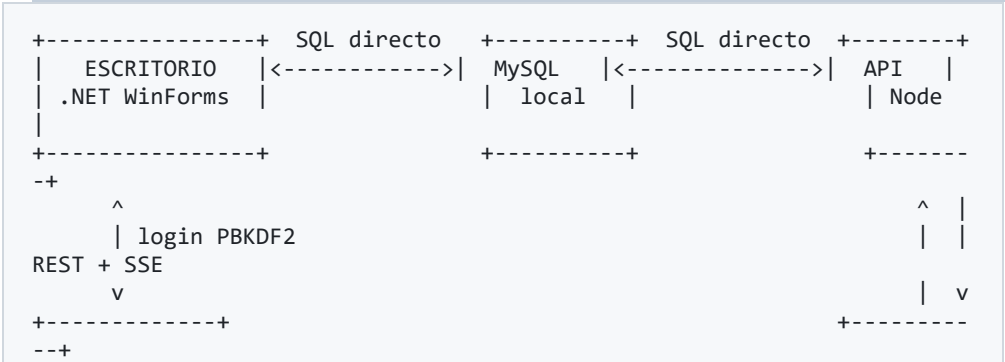
- 1. Mapa general del sistema
- 2. El aplicativo de escritorio: MVC en .NET WinForms
- 3. El apartado grafico del escritorio (en detalle)
- 4. La API: el puente entre los dos mundos
- 5. La app movil: React y su flujo de datos
- 6. El sistema entero en movimiento (3 recorridos reales)
- 7. Limites conocidos y glosario

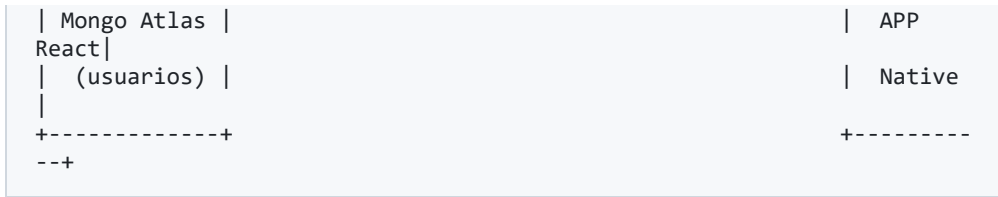
1. Mapa general del sistema

El proyecto tiene **tres piezas de software** y **dos bases de datos**. Lo esencial es esto: el MySQL local es la unica fuente de verdad compartida. El escritorio y la API leen y escriben la misma base; la app movil nunca toca la base directamente, solo habla con la API por HTTP.

Pieza	Tecnologia	Donde vive	Que hace
Escritorio (SistemaAsistencia)	.NET Framework 4.7.2, WinForms, MaterialSkin	PC con Windows	ABMs completos: alumnos, profesores, preceptores, materias, especialidades, dictados, inscripciones, usuarios. Escribe directo a MySQL.
API (Api)	Node.js, Express 5, mysql2, bcryptjs	Servidor local, puerto 3000	Expone REST + eventos en tiempo real (SSE). La usa solo la app movil.
App (App)	React Native (Expo SDK 57), React 19	Celular / navegador	Login por DNI, abrir/cerrar clases con QR, tomar y verificar asistencia.

Base	Motor	Contenido	Quien la usa
gestion_asistencia_eest	MySQL local	alumno, profesor, preceptor, materia, especialidad, dictado, inscribe, clase, asistencia	Escritorio y API (ambos, directo)
MongoDB Atlas	MongoDB en la nube	Coleccion de Usuario (login del escritorio)	Solo el escritorio





Idea clave: hay dos caminos de escritura hacia el MySQL (escritorio directo y app via API) pero un solo camino de notificaciones en tiempo real (la API por SSE). Todo cambio que nace en el escritorio llega a la base pero **no genera evento**; la app lo ve recién cuando vuelve a consultar.

2. El aplicativo de escritorio: MVC en .NET WinForms

El escritorio esta organizado en capas con responsabilidades estrictas. La regla de oro: **la Vista nunca toca la base; el Controlador nunca arma SQL; el Modelo nunca dibuja nada.**

```
Vista (Forms)  --llama-->  Controlador  --llama-->  DAO (Modelo)  --
SQL-->  MySQL
      ^
      |
      |___ muestra objetos      | Ejecutor + permisos      | ___ devuelve
objetos                        |                          |      (Entidades)
      simples (List<T>)      por rol
```

2.1 Arranque: que pasa al hacer doble clic

Todo empieza en `Program.cs`, metodo `Main()`. El orden es fijo:

1. `Application.EnableVisualStyles()`: activa el dibujado moderno de Windows.
2. Si no hay configuracion guardada, se abre `FrmConfiguracion` como dialogo modal (`ShowDialog()` bloquea hasta que el usuario acepta o cancela; si cancela, el programa termina).
3. Si no existen usuarios en Mongo, se abre `FrmPrimerUsuario` para crear el primer administrador.
4. `Application.Run(new FrmLogin())`: arranca el bucle de mensajes de Windows con el login como ventana principal.
5. Todo el arranque esta envuelto en `try/catch`: cualquier error se guarda con `Logger` y se muestra un `MessageBox`.

2.2 El Modelo: entidades, DAOs y conexiones

Entidades: objetos simples

Cada tabla tiene su clase POCO (solo propiedades, sin logica). Ejemplo real (`Modelo/Entidades/Preceptor.cs`):

```
public class Preceptor
{
    public int IdPreceptor { get; set; }
    public string NombrePreceptor { get; set; }
    public string Dni { get; set; }
    public string Contraseña { get; set; }
    public bool Activo { get; set; }
    public string NombreCompleto => $"{ApellidoPreceptor},
{NombrePreceptor}";
```

```
}
```

La propiedad calculada `NombreCompleto` existe solo para mostrar lindo en grillas y listas, no se guarda en la base.

DAOs: el unico lugar donde hay SQL

Cada DAO (ej. `PreceptorDAO`) implementa el CRUD con reglas fijas del proyecto:

- **Conexion por operacion:** cada metodo abre su `MySqlConnection` dentro de un `using` (se cierra sola) usando `ConexionBD`, que lee la cadena "MySQL" del `App.config`.
- **SQL parametrizado y en mayusculas** (`@nombre`, `@dni`), nunca concatenado: evita inyeccion SQL.
- **Baja logica:** los listados filtran `WHERE activo = 1` y "eliminar" es `UPDATE ... SET activo = 0`. No se borra el historico.
- **Mapeo manual:** un metodo privado (`MapearPreceptor`) convierte cada fila del `MySqlDataReader` en un objeto.
- **Duplicados:** si MySQL devuelve error 1062 (clave duplicada), se traduce a `DatosException("Ya existe un preceptor con ese DNI o legajo.")`.
- **Eliminacion definitiva transaccional:** borra dictados vinculados + preceptor dentro de una transaccion (todo o nada).
- **Contraseñas con bcrypt:** al crear, `Crypter.Blowfish.Crypt(plana, GenerateSalt(10))` guarda un hash `$2a$10$...` identico en formato al que verifica la API. Al modificar, si el campo viene vacio se conserva el hash actual (`IFNULL(@contrasena, contrasena)`).

Dos conexiones, dos mundos

- `ConexionBD` (MySQL local): todos los datos de negocio.
- `ConexionMongo` (Atlas): solo la coleccion `Usuario` para el login del escritorio, con hash PBKDF2 (prefijo `pbkdf2$`, 10000 iteraciones). Es un esquema de autenticacion distinto al de la API (que usa DNI + bcrypt en MySQL).

2.3 Los Controladores: fachada, errores y permisos

El controlador es una fachada delgada: recibe objetos de la Vista, delega al DAO y devuelve resultados. Ejemplo real (`Controlador/PreceptorController.cs`):

```
public List<Preceptor> ObtenerPreceptores()
{
    return Ejecutor.Ejecutar("Preceptor.ObtenerPreceptores",
        () => preceptorDAO.ObtenerTodos());
}

public bool EliminarDefinitivo(int idPreceptor)
{
}
```

```

        if (!Sesion.UsuarioActualRolAdministrador())
            throw new DatosException("Solo el administrador
puede...");
        return Ejecutor.Ejecutar("Preceptor.EliminarDefinitivo",
            () => preceptorDAO.EliminarDefinitivo(idPreceptor));
    }

```

Tres piezas transversales sostienen a todos los controladores:

- **Ejecutor.Ejecutar(contexto, accion):** ejecuta la operacion; si falla con un error tecnico lo registra en el log y lo convierte en **DatosException** con mensaje amigable. Si ya era **DatosException** (ej. "DNI obligatorio"), la deja pasar intacta.
- **DatosException:** la unica excepcion que las Vistas saben mostrar al usuario.
- **Sesion:** clase estatica con **UsuarioActual**. Las operaciones peligrosas preguntan **UsuarioActualRolAdministrador()** antes de ejecutarse.

2.4 Las Vistas: formularios que solo piden y muestran

Cada ABM es un **Form** (ej. **Vista/Preceptores/FrmPreceptores.cs**) que sigue siempre el mismo guion: llama al controlador dentro de **try/catch** (**DatosException**), muestra el resultado en una grilla o un mensaje, y valida los campos antes de enviar (ej. DNI obligatorio y solo digitos). La Vista jamas arma SQL ni decide permisos de negocio.

El formulario principal (**Vista/Principal/FrmPrincipal.cs**) es un **shell**: menu lateral con una opcion por modulo (Inicio, Usuarios, Alumnos, Profesores, Preceptores, Materias, Especialidades, Dictados, Inscripciones), cada una con su color de acento. Al elegir una opcion, el formulario correspondiente se **incrusta** en el panel central:

```

form.TopLevel = false;           // deja de ser ventana
independiente
form.Dock = DockStyle.Fill;     // ocupa todo el panel
panelContenido.Controls.Add(form);

```

Encima se dibuja una "cinta" del color del modulo. **AplicarPermisos()** oculta opciones segun el rol, y la barra inferior muestra usuario y rol de **Sesion**.

Login del escritorio

FrmLogin pide usuario y clave, llama a **UsuarioController.IniciarSesion()** (verifica el hash PBKDF2 en Mongo), guarda en **Sesion.UsuarioActual**, oculta el login y abre **FrmPrincipal** con **ShowDialog()**. Cerrar sesion vuelve a mostrar el login.

2.5 Recorrido completo: alta de un preceptor

1. El usuario completa nombre, apellido, legajo, DNI (y opcionalmente contraseña) en FrmPreceptores y pulsa Guardar.
2. La Vista valida (DNI presente y numerico) y arma un objeto Preceptor.
3. Llama a PreceptorController.AgregarPreceptor(), que lo envuelve con Ejecutor.
4. El DAO hashea la contraseña (o el DNI si vino vacia) con bcrypt coste 10 y ejecuta el INSERT parametrizado.
5. Si el DNI o legajo ya existen (error 1062), sube una DatosException que la Vista muestra en un cartel; si no, la grilla se recarga con ObtenerPreceptores().
6. Ese preceptor ya puede loguearse en la app movil con su DNI, porque la API verifica el mismo formato bcrypt.

3. El apartado grafico del escritorio (en detalle)

3.1 WinForms + MaterialSkin: de que se trata

Windows Forms dibuja ventanas classicas de Windows con controles (botones, campos de texto, grillas). El proyecto usa la libreria **MaterialSkin.2**, que reemplaza el aspecto por el lenguaje visual "Material" de Google sin cambiar la programacion: los formularios heredan de `MaterialForm` y usan controles `MaterialTextBox`, `MaterialButton`, etc. El tema se configura una vez por formulario con un singleton:

```
MaterialSkinManager skin = MaterialSkinManager.Instance;
skin.AddFormToManage(this);
skin.Theme = MaterialSkinManager.Themes.LIGHT;
skin.ColorScheme = new ColorScheme(Primary.Indigo600, ...);
```

3.2 El Designer: codigo que escribe codigo

Cada formulario tiene un archivo companero `.Designer.cs` generado por el editor visual: crea cada control, le fija posicion, tamano, colores y lo agrega con `Controls.Add(...)`. El programador casi nunca lo toca a mano; dibuja en el editor y Visual Studio escribe ese codigo. Por eso un cambio visual (agregar el campo DNI) implica regenerar o reescribir ese archivo con coordenadas exactas.

3.3 Eventos y dibujado manual

WinForms es **dirigido por eventos**: cada clic o cambio dispara un metodo (`btnGuardar_Click`, `FormClosing`, `Shown`). El hilo principal corre un bucle de mensajes (`Application.Run`) que reparte esos eventos; por eso las consultas a la base se hacen sincronicas y rapidas, para no congelar la ventana. Detalles finos reales del proyecto: la ventana principal evita el maximizado emulado de MaterialSkin en multi-monitor (una TV con otra resolucion cortaba la parte de abajo) y en su lugar ocupa manualmente el area de trabajo del monitor donde esta el cursor; y al cerrar con la cruz se fuerza `Environment.Exit(0)` salvo que venga de "Cerrar sesion".

3.4 Grillas y dialogos

- Las listas se muestran en grillas enlazadas a `List<T>` (enlace manual: se asigna la lista y la grilla lee las propiedades).
- Las confirmaciones peligrosas usan dialogos modales (`FrmConfirmarEliminar`, que muestra dependencias como "Dictados

a cargo: 3") y el boton de eliminacion definitiva solo es visible para el administrador.

- Los mensajes usan `MaterialMessageBox` (el cartel con estilo Material) en vez del clasico.

4. La API: el puente entre los dos mundos

La API (`Api/index.js`, Express 5, puerto 3000) expone REST y un canal de eventos. Sus rutas reales:

Ruta	Metodo	Rol	Hace
<code>/api/login</code>	POST	publico	Busca por DNI en preceptor/profesor/alumno con <code>activo = 1</code> , compara bcrypt, devuelve token + usuario.
<code>/api/eventos</code>	GET (SSE)	todos	Canal de eventos en tiempo real.
<code>/api/dictados</code>	GET x3	publico	Lista dictados, dictados de un alumno, alumnos de un dictado.
<code>/api/clases</code>	GET/POST/PATCH + token + escanear	segun accion	Abrir clase (profesor), cerrar (preceptor), rotar QR (profesor), escanear (alumno).
<code>/api/asistencia</code>	GET/POST/lote	preceptor	Consultar y guardar asistencia individual o en lote.

4.1 Autenticacion: token firmado casero

Al loguearse, `crearToken()` arma `payload.firma`: el payload (rol, dni, ids, vencimiento a 8 h) en base64url, firmado con HMAC-SHA256 y una clave secreta. Cada pedido lleva `Authorization: Bearer <token>`; el middleware `requiereRol([...])` verifica la firma en tiempo constante, controla el vencimiento y el rol (401 sin sesion, 403 sin permiso).

4.2 Tiempo real: SSE

`utilidades/eventos.js` mantiene un `Set` de clientes conectados. `publicarEvento()` guarda el evento en un historial (ultimos 100, numerados) y lo escribe a cada cliente autorizado en formato `id: / event: / data: .` Hay tres mecanismos de robustez: latido : heartbeat cada 25 s, retry: 3000 (el cliente reintenta a los 3 s si se cae) y reenvio del historial con `Last-Event-ID` al

reconectar. Además filtra por rol y, para alumnos, solo les entrega eventos de su propio `id_alumno`. Los eventos existen solo para acciones que pasan por la API (`clase.abierta`, `clase.cerrada`, `qr.rotado`, `asistencia.registrada`, etc.): lo que el escritorio escribe directo en MySQL no publica nada.

5. La app movil: React y su flujo de datos

5.1 React en cinco ideas

1. **Componentes:** la pantalla se arma con funciones que devuelven interfaz (ej. `function ProfesorPrincipal(...)`). Cada componente es una pieza reusable.
2. **JSX:** se escribe HTML-like (`<View><Text>Hola</Text></View>`) mezclado con JavaScript entre llaves (`{sesion.nombre}`). No es un template aparte: es código.
3. **Props:** datos que un padre pasa al hijo (`<ProfesorPrincipal sesion={sesion} evento={evento} />`). Fluyen en una sola dirección: de arriba hacia abajo.
4. **Estado (useState):** la memoria del componente (`const [clase, setClase] = useState(null)`). Llamar a `setClase(...)` le dice a React "algo cambio, redibuja".
5. **Render y reconciliación:** ante cada cambio de estado o props, React vuelve a ejecutar la función, compara el resultado con lo anterior y actualiza solo lo que cambio en pantalla. El programador describe *como se ve cada estado*; React se ocupa del *como actualizar*.

5.2 React Native + Expo: React sin navegador

En vez de `<div>` y CSS hay `<View>`, `<Text>`, `<ScrollView>` y estilos con `StyleSheet.create({...})` (objetos JavaScript con la misma lógica que CSS: flex, colores, márgenes). React Native traduce esos componentes a vistas **nativas** del celular. Expo (SDK 57) es la plataforma que empaqueta, ejecuta (`expo start`) y da acceso a hardware: cámara (`expo-camera`, para escanear el QR), QR dibujado (`react-native-qrcode-svg`) y constantes del entorno (`expo-constants`, de donde sale la IP del servidor para armar `http://<host>:3000`). La app corre en Android/iOS y también en web.

5.3 Arranque y sesion (App.js)

`index.js` registra el componente raíz. `App.js` guarda en estados: `sesion`, campos de login, cargando, mensaje, último evento y eventosConectados. Sin `sesion` muestra el login (con cuentas de prueba en modo desarrollo); al ingresar, guarda el usuario y renderiza la pantalla según `sesion.rol`. Un `useEffect` atado a `[sesion?.dni, sesion?.rol]` abre el canal SSE; al cerrar sesión o cambiar de usuario, la limpieza del efecto anterior corta la conexión vieja antes de abrir la nueva. El encabezado muestra "Sincronización en tiempo real" o "Usando polling de respaldo" según el estado del canal.

5.4 La capa de red (src/api.js)

- El token vive en una variable del modulo (`tokenSesion`); `pedir()` hace `fetch` agregando el Bearer, parsea JSON y lanza `Error` si la respuesta no es OK. `post()/patch()` son atajos.
- `conectarEventos(alEvento, alCambiarEstado)` abre el SSE con `fetchEventSource`: valida que la respuesta sea un canal valido, ignora mensajes vacios, parsea el JSON y avisa cada evento; ante cierre o error pide reintento en 3 s (salvo errores fatales 4xx). Devuelve una funcion de limpieza que aborta la conexion (`AbortController`).

5.5 Flujo de datos por pantalla

Profesor (`ProfesorPrincipal.js`): carga sus dictados y filtra por su `id_profesor`; al abrir una clase guarda el objeto `clase` (con token QR) en estado y dibuja el QR (`{id_clase, token}`) con cuenta regresiva de vencimiento. Cada evento `clase.abierta/cerrada` o `qr.rotado` de su clase dispara una recarga puntual. Ademas hay reconciliacion cada 30 s con SSE (o cada 5 s sin SSE, solo si la app esta activa). Acciones: abrir clase y rotar token.

Preceptor (`PreceptorPrincipal.js`): lista clases, selecciona una, carga sus alumnos y edita un mapa de estados (`{id_alumno: 'presente'/'verificada'}`); guarda en lote y puede cerrar la clase. Reacciona a `asistencia.registrada/actualizada` y a apertura/cierre de clases.

Alumno (`AlumnoPrincipal.js`): lista sus dictados, abre la camara (`CameraView`), escanea el QR y llama a `escanearClase`; reacciona a apertura, cierre, rotacion de QR y registro de su asistencia.

5.6 Efectos y ciclo de vida

`useEffect(fn, [deps])` ejecuta `fn` cuando cambian las dependencias y corre su `return` (limpieza) antes de repetir o al desmontar: asi se abren/cierran conexiones e intervalos sin fugas. `useCallback` evita recrear funciones para no disparar efectos de mas. Patron repetido en las tres pantallas: *efecto de carga inicial + efecto que reacciona al ultimo evento + intervalo de respaldo + limpieza siempre*.

6. El sistema entero en movimiento

Recorrido A: login en el movil

La app envia DNI + contraseña a `POST /api/login` → la API busca en las tres tablas con `activo = 1`, compara bcrypt, firma el token y lo devuelve → `api.js` lo guarda en memoria → `App.js` guarda el usuario en estado y abre el SSE y la pantalla del rol.

Recorrido B: una clase con QR, de punta a punta

1. El profesor abre la clase (`POST /api/clases`) → la API inserta en MySQL y publica `clase.abierta`.
2. Preceptor y alumnos reciben el evento y recargan; el profesor ve el QR con el token.
3. El alumno escanea (`POST .../escanear`) → la API valida el token, inserta la asistencia y publica `asistencia.registrada`.
4. El preceptor verifica en persona, guarda el lote y cierra (`PATCH ... cerrada`) → evento `clase.cerrada` y todos se actualizan.

Recorrido C: un cambio desde el escritorio

El preceptor a cargo de un dictado se cambia en `FrmDictados` → `UPDATE` directo en MySQL → **no se publica ningun evento**. La app del profesor lo vera recién en su proxima consulta (reconciliacion o re-login). Este es el limite arquitectonico actual y el motivo de la propuesta de sincronizacion (outbox + triggers).

7. Limites conocidos y glosario

Tema	Estado
SSE escritorio → app	No existe: el escritorio no emite eventos (ver Recorrido C).
Login API vs bajas logicas	Resuelto: filtra <code>activo = 1</code> .
Contraseñas entre mundos	Resuelto: CryptSharp (escritorio) y bcryptjs (API) son interoperables, coste 10.
Identidad del escritorio	Mongo + PBKDF2, separada de la API; el rol Administrador no existe en la API.
Compilacion del escritorio	Solo con el MSBuild de Visual Studio 18; <code>dotnet msbuild</code> falla en este proyecto clasico.
Credenciales hardcodeadas	Intencional por consigna del curso (ver AGENTS.md); no modificar.

Glosario minimo

- **MVC:** Modelo (datos) - Vista (pantalla) - Controlador (intermediario). En el escritorio: DAO - Form - *Controller.
- **DAO:** objeto cuya unica mision es hablar con la base (SQL).
- **Baja logica:** "borrar" poniendo `activo = 0` en vez de DELETE.
- **SSE:** canal HTTP unidireccional servidor → cliente para eventos en tiempo real.
- **JWT/HMAC casero:** token `payload.firma` que prueba identidad sin consultar la base en cada pedido.
- **Props/Estado (React):** datos que bajan del padre / memoria local que al cambiar redibuja.
- **MaterialSkin:** libreria que viste los WinForms con estilo Material Design.

Archivos clave por si quieres leer el codigo

- Escritorio:
`.net/SistemaAsistencia/SistemaAsistencia/Program.cs,`
`Modelo/DAO/*.cs,` `Controlador/*.cs,`

Vista/Principal/FrmPrincipal.cs,
Utilidades/{Sesion,Ejecutor,DatosException}.cs.

- API: Api/index.js, Api/rutas/*.js,
Api/utilidades/{autenticacion,eventos}.js.
- App: App/App.js, App/src/api.js, App/src/pantallas/*.js.